



THE ICT UNIVERSITY

IMPARTING ICTs IN ALL ACADEMIC DISCIPLINES

Year 3 Software Design and Implementation Project

AgriConnect

Agricultural Management Platform

“Connecting Farms to Markets”

Team Members

[NJOYA MEDIN PRAISE ALAMBI]

Scrum

Master

/

Back-

end

Frontend

De-

vel-

oper

[NGEMINANG PRECIOUS]

Project Details

Course: Software Design and Implementation

Instructor: [ENGR PALMA]

Institution: [THE ICT UNIVERSITY]

Date: January 22, 2026

Contents

1	Abstract	1
2	Introduction	1
3	Problem Overview	3
3.1	Challenges Faced by Farmers	3
3.2	Challenges Faced by Buyers	3
3.3	Limitations of Existing Solutions	4
3.4	Motivation for AgriConnect	4
4	System Requirements	4
5	Technology Stack	5
5.1	Frontend	5
5.2	Backend	5
5.3	Database & Caching	6
5.4	Containerization & Orchestration	6
5.5	DevOps & CI/CD	6
5.6	Why This Stack?	6
5.7	Directory Structure Overview	7
5.8	Functional Requirements	7
5.8.1	User Registration and Authentication	8
5.8.2	User Profile Management	8

5.8.3	Farm Record Management	8
5.8.4	Product Listing and Management	8
5.8.5	Market Access and Buyer Interaction	8
5.9	Non-Functional Requirements	9
5.9.1	Usability	9
5.9.2	Performance	9
5.9.3	Security	9
5.9.4	Scalability	9
5.9.5	Reliability and Availability	9
5.10	User-Specific Requirements	9
5.10.1	Farmer Requirements	10
5.10.2	Buyer Requirements	10
5.11	System Constraints	10
6	System Architecture	10
6.1	Architecture Overview	11
7	System Architecture	11
7.1	Architecture Overview	11
7.2	Client Layer	12
7.3	Server Layer	12
7.4	Database Layer	13
7.5	Justification for Load Balancer Inclusion	13
7.6	Comparison with Alternative Architectures	13
7.7	Future Considerations	13

8	System Design	14
8.1	Use Case Modeling	14
8.1.1	Explanation of Use Cases	14
8.2	Class Diagram Modeling	15
8.2.1	Explanation of Classes	15
8.3	Database Design	16
8.3.1	Database Explanation	16
8.4	User Interface Design	17
8.5	Design Justification	17
9	Implementation	18
9.1	Technology Stack	18
9.2	Module-to-Code Mapping	19
9.3	Example Implementation Snippets	19
9.3.1	User Authentication (Backend)	19
9.3.2	Product Listing (Frontend)	20
9.4	Integration Strategy	20
9.5	Best Practices Applied	21
10	Testing	21
10.1	Testing Strategy	21
10.2	Test Cases	22
10.3	Testing Tools	22
10.4	Testing Results and Analysis	23
10.5	Conclusion on Testing	23

11 Deployment	23
11.1 Deployment Architecture	23
11.2 Frontend Deployment on Vercel	24
11.3 Backend Deployment on AWS with ELB	25
11.4 Continuous Integration and Deployment	25
11.5 Security Considerations	25
11.6 Scalability and Reliability	26
12 Conclusion	26

List of Figures

1	AgriConnect Client-Server Architecture with Load Balancer	12
2	Use Case Diagram for AgriConnect (Farmer and Buyer Interactions)	14
3	Class Diagram of AgriConnect (Core System Classes and Relationships)	15
4	Entity-Relationship Diagram for AgriConnect Database	16
5	Example User Interface of AgriConnect	17
6	Deployment Architecture of AgriConnect using AWS ELB and Vercel	24

List of Tables

1	Sample Test Cases for AgriConnect	22
---	---	----

1 Abstract

AgriConnect is a web-based agricultural management platform designed to bridge the gap between farmers and buyers. The platform facilitates farm record management, product listing, communication between users, and market browsing. This report provides detailed documentation of the project, including requirements analysis, system architecture, design, implementation, testing, and deployment. The system leverages modern technologies such as React.js, Node.js, PostgreSQL, AWS, and Vercel to ensure scalability, security, and usability.

2 Introduction

Agriculture is one of the most fundamental sectors of human society, providing food, raw materials, and employment for a significant proportion of the world's population. In developing countries, agriculture serves not only as a source of livelihood but also as a backbone of economic stability and rural development. Despite its importance, the agricultural sector continues to face numerous challenges that limit productivity, efficiency, and profitability, particularly among smallholder farmers.

Small-scale farmers constitute the majority of agricultural producers in many African countries. These farmers typically operate with limited resources, rely heavily on manual processes, and often lack access to modern technologies that could improve their operations. One of the most pressing challenges faced by farmers is limited access to reliable and profitable markets. Agricultural produce is often sold through intermediaries who dictate prices, leaving farmers with minimal profit margins. The lack of transparent pricing mechanisms further exacerbates this issue, making it difficult for farmers to make informed decisions regarding production and sales.

Another major challenge within the agricultural sector is poor farm record management. Many farmers still rely on manual record keeping or do not keep records at all. As a result, tracking crop yields, expenses, seasonal patterns, and farm performance becomes extremely difficult. This lack of structured data negatively impacts planning, resource allocation, and long-term sustainability. Without accurate records, farmers are unable to evaluate the success of previous farming cycles or adopt data-driven approaches to improve productivity.

In addition to market access and record management issues, communication gaps also hinder agricultural growth. Farmers often lack direct communication channels with buyers, suppliers, and agricultural experts. Information such as demand trends, quality requirements, and market

expectations is rarely communicated effectively. This disconnect leads to post-harvest losses, unsold produce, and reduced income.

The rapid advancement of information and communication technologies presents an opportunity to address many of these challenges. Digital platforms have proven effective in improving efficiency, transparency, and connectivity across various sectors. In agriculture, technology-driven solutions can facilitate market access, improve information flow, and provide tools for better farm management. Mobile and web-based applications, in particular, offer scalable solutions that can reach a wide audience, including rural communities.

AgriConnect was conceived as a response to the challenges identified within the agricultural value chain. The platform is designed as a digital agricultural management system that connects farmers directly to buyers while providing essential tools for managing farm activities. AgriConnect enables farmers to create digital profiles, manage farm records, and list agricultural produce for sale on an accessible platform. Buyers, on the other hand, can browse available products, view farmer details, and communicate directly with producers.

A key motivation behind the development of AgriConnect is the reduction of dependency on intermediaries. By enabling direct interaction between farmers and buyers, the platform promotes transparency and fair pricing. Farmers gain greater control over how their produce is marketed and sold, while buyers benefit from access to fresh produce and reliable sources.

The system is designed with usability and accessibility as core priorities. Recognizing that many users may have limited experience with digital platforms, AgriConnect emphasizes simple navigation, clear interfaces, and straightforward workflows. The platform is also designed to be scalable, allowing for future expansion to include additional features such as mobile application support, online payment systems, and agricultural advisory services.

This project focuses on the design and implementation of the AgriConnect platform using standard software engineering principles. The development process includes requirement analysis, system architecture design, implementation, testing, and deployment. Through this structured approach, the project aims to demonstrate how well-designed software systems can address real-world agricultural challenges and contribute to sustainable development.

The remainder of this document is organized as follows: the system requirements are discussed in the next section, followed by a detailed explanation of the system architecture. Subsequent sections present the system design, implementation details, testing strategies, deployment process, and finally, a conclusion with proposed future enhancements.

3 Problem Overview

Agriculture remains a backbone of many economies, particularly in developing countries, yet farmers often encounter numerous challenges that hinder productivity and profitability. Despite the proliferation of technology in various sectors, small to medium-scale farmers frequently lack efficient systems to manage farm operations and connect with buyers. AgriConnect was developed to address several key issues identified in the agricultural sector.

3.1 Challenges Faced by Farmers

Farmers face difficulties in managing their daily farm activities due to limited access to digital tools. Common challenges include:

- **Record-Keeping:** Many farmers rely on paper-based systems to track crop types, planting schedules, and yields, leading to errors and inefficiencies.
- **Market Access:** Farmers often struggle to reach potential buyers directly, relying on intermediaries who reduce their profit margins.
- **Price Discovery:** Without a centralized platform, farmers cannot easily compare prices across markets, which may result in selling products below fair value.
- **Communication Barriers:** Lack of direct communication channels between farmers and buyers slows down transactions and can lead to misunderstandings or lost sales.
- **Knowledge Sharing:** Farmers have limited access to timely agricultural best practices, pest control information, and weather alerts, affecting productivity.

3.2 Challenges Faced by Buyers

Similarly, buyers face issues that prevent efficient sourcing of agricultural products:

- **Limited Supplier Visibility:** Buyers struggle to find reliable suppliers and often depend on local networks, which may be inconsistent or unreliable.
- **Product Quality Verification:** Without digital platforms, it is challenging for buyers to verify farm product quality before purchase.
- **Delayed Communication:** Communication with multiple farmers can be slow and fragmented, resulting in delayed procurement.
- **Inefficient Logistics Planning:** Lack of structured data on availability and location of farm products increases logistical costs and complexity.

3.3 Limitations of Existing Solutions

While some agricultural management systems exist, they are often limited in scope or accessibility:

- Mobile apps may only focus on record-keeping without connecting farmers to buyers.
- Market platforms often do not provide farm-level data or transaction tracking.
- Existing platforms may not be accessible to farmers in rural areas due to internet connectivity or high costs.

3.4 Motivation for AgriConnect

AgriConnect was designed to bridge these gaps by providing an integrated platform that combines farm management, product listing, buyer-farmer communication, and market discovery. By leveraging modern web technologies, cloud deployment, and a user-friendly interface, AgriConnect aims to:

- Streamline farm record management digitally
- Connect farmers directly to buyers, reducing intermediaries
- Improve price transparency and market access
- Enhance communication, coordination, and knowledge sharing
- Provide a scalable and reliable platform that can grow with user demand

This problem overview establishes the need for AgriConnect and sets the context for the system requirements, architecture, and design discussed in subsequent sections.

4 System Requirements

This section describes the requirements of the AgriConnect system. System requirements define what the system is expected to do and the constraints under which it must operate. These requirements were derived from an analysis of the challenges faced by farmers and buyers within the agricultural value chain, as well as the overall objectives of the project.

The requirements are categorized into functional requirements, non-functional requirements, user-specific requirements, and system constraints to provide a comprehensive understanding of the system's expected behavior.

5 Technology Stack

AgriConnect leverages a modern, scalable, and containerized technology stack to ensure high performance, maintainability, and ease of deployment. The stack integrates frontend, backend, database, caching, containerization, and DevOps practices as described below.

5.1 Frontend

- **Framework:** *Next.js* — provides server-side rendering, static site generation, and efficient routing.
- **Language:** *JavaScript (ES6+)* and *TypeScript* for type safety.
- **Styling:** *TailwindCSS* for responsive and modern UI components.
- **API Communication:** *Fetch API* / *Axios* for interacting with the backend RESTful endpoints.
- **Deployment:** *Vercel* is used to host the frontend, providing continuous integration, HTTPS endpoints, and CDN support.

5.2 Backend

- **Framework:** *Django* (Python) — robust and scalable backend framework, providing REST APIs via Django REST Framework (DRF).
- **Web Server:** *Gunicorn* — production WSGI server for handling concurrent requests.
- **Task Queue:** *Celery* with *Redis* for asynchronous tasks (notifications, analytics processing).
- **Authentication:** *JWT* (JSON Web Tokens) for secure user sessions.
- **Deployment:** Hosted on *AWS EC2* instances behind an *Elastic Load Balancer* (ELB).

5.3 Database & Caching

- **Primary Database:** PostgreSQL hosted on AWS RDS — ensures relational data integrity for users, farms, products, and transactions.
- **Caching:** Redis — used for caching frequently accessed data and handling asynchronous tasks via Celery.
- **Persistence:** Volumes are mounted via Docker to persist database and media files.

5.4 Containerization & Orchestration

- **Docker:** Provides isolated, consistent environments for frontend, backend, database, and cache services.
- **Docker Compose:** Simplifies local development by orchestrating multi-container services.
- **Multi-stage Docker Builds:** Optimized production images for both frontend and backend to reduce image size and improve deployment efficiency.
- **Optional Production Orchestration:** Docker Swarm or Kubernetes for horizontal scaling, service discovery, and high availability.
- **Reverse Proxy:** Nginx or Traefik as a load balancer and SSL termination point.

5.5 DevOps & CI/CD

- **CI/CD:** GitHub Actions, GitLab CI, or Jenkins pipelines to automate testing, building, and deployment.
- **Environment Management:** .env files for development and production environment variables, Docker secrets for sensitive data.
- **Monitoring & Logging:** Docker logs, Prometheus, and Grafana for performance and health monitoring.

5.6 Why This Stack?

The chosen technology stack was selected to achieve the following objectives:

- **Scalability:** Load balancers, containerization, and AWS infrastructure allow horizontal scaling as user demand grows.
- **Consistency:** Docker ensures the same environment across development, testing, and production.
- **Rapid Development:** Modern frameworks (Next.js, Django) and CI/CD pipelines accelerate development and deployment cycles.
- **Maintainability:** Clear separation of frontend, backend, and database components enables modular updates and debugging.
- **Security:** JWT authentication, HTTPS endpoints, and managed cloud services ensure user data protection.

5.7 Directory Structure Overview

```
agri-connect/  
|-- docker-compose.yml  
|-- .dockerignore  
|-- frontend/  
|   |-- Dockerfile  
|   '-- ...  
|-- backend/  
|   |-- Dockerfile  
|   |-- entrypoint.sh  
|   '-- ...  
 '-- nginx/ (optional)  
     '-- nginx.conf
```

This directory structure, combined with Docker containerization, ensures easy setup for new developers, isolated environments for testing, and smooth deployment to production servers.

5.8 Functional Requirements

Functional requirements describe the specific services and operations that the AgriConnect system must provide. These requirements define how users interact with the system and the core functionalities that support agricultural management and market connectivity.

5.8.1 User Registration and Authentication

- The system shall allow users to register as either farmers or buyers.
- The system shall provide secure login functionality using unique credentials.
- The system shall allow users to log out of the platform securely.
- The system shall ensure that only authenticated users can access protected features.

5.8.2 User Profile Management

- The system shall allow users to create and update personal profiles.
- Farmers shall be able to include farm-related information such as location and type of produce.
- Buyers shall be able to update contact and preference information.

5.8.3 Farm Record Management

- The system shall allow farmers to create digital records of farming activities.
- The system shall enable farmers to record crop type, planting date, and expected harvest time.
- The system shall allow farmers to update and view historical farm records.

5.8.4 Product Listing and Management

- The system shall allow farmers to list agricultural products for sale.
- Each product listing shall include product name, quantity, price, and availability status.
- The system shall allow farmers to update or remove product listings.

5.8.5 Market Access and Buyer Interaction

- The system shall allow buyers to browse available agricultural products.
- The system shall allow buyers to search and filter products based on criteria such as product type and location.
- The system shall enable direct communication between buyers and farmers.

5.9 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. These requirements ensure that AgriConnect performs efficiently, securely, and reliably under various conditions.

5.9.1 Usability

- The system shall provide a simple and intuitive user interface.
- The system shall be easy to use for users with limited technical knowledge.
- Navigation within the system shall be clear and consistent.

5.9.2 Performance

- The system shall respond to user actions within an acceptable time.
- The system shall support multiple users accessing the platform simultaneously.

5.9.3 Security

- The system shall protect user data from unauthorized access.
- The system shall ensure secure handling of authentication credentials.
- The system shall restrict access to sensitive data based on user roles.

5.9.4 Scalability

- The system shall be able to accommodate an increasing number of users.
- The system shall support future feature expansion without major redesign.

5.9.5 Reliability and Availability

- The system shall be available for use with minimal downtime.
- The system shall handle system errors gracefully without data loss.

5.10 User-Specific Requirements

Different users interact with the system in different ways. The AgriConnect platform identifies two primary user roles: farmers and buyers.

5.10.1 Farmer Requirements

- Farmers shall be able to manage farm and product information.
- Farmers shall have control over product pricing and availability.
- Farmers shall be able to communicate directly with interested buyers.

5.10.2 Buyer Requirements

- Buyers shall be able to view available agricultural products.
- Buyers shall be able to contact farmers for negotiation or inquiries.
- Buyers shall be able to evaluate product details before purchase.

5.11 System Constraints

System constraints define limitations under which the AgriConnect system must operate.

- The system shall require an internet connection to function.
- The system shall operate on standard web browsers.
- The system shall be developed within the academic project timeline.
- The system shall utilize available development tools and technologies.

6 System Architecture

System architecture defines the high-level structure of a software system, describing how different components are organized and how they interact with one another. A well-designed architecture is essential for ensuring that a system is scalable, maintainable, secure, and capable of meeting both current and future requirements. For the AgriConnect platform, careful consideration was given to selecting an architecture that could support a growing number of users while remaining simple enough for efficient development and maintenance.

6.1 Architecture Overview

AgriConnect adopts a multi-tier (layered) architecture, which separates the system into distinct layers, each with a specific responsibility. The architecture consists of three primary layers: the presentation layer, the application (business logic) layer, and the data layer. This architectural approach promotes separation of concerns, allowing each layer to evolve independently without affecting the overall system stability.

The layered architecture was selected because it aligns well with modern web-based applications and provides a clear structure for managing complexity. By isolating user interaction, business logic, and data management, the system becomes easier to understand, test, and extend.

7 System Architecture

System architecture defines the high-level structure of a software system, describing how components interact to deliver required functionalities. A well-designed architecture ensures scalability, maintainability, and security. For AgriConnect, a **Client-Server architecture** was selected due to its suitability for web-based applications and centralized control requirements.

7.1 Architecture Overview

The Client-Server architecture divides the system into primary entities: clients, a server cluster, and the database. To support scalability and high availability, a **load balancer** is introduced between clients and backend servers. The load balancer distributes incoming requests across multiple server instances, ensuring no single server becomes a bottleneck and improving fault tolerance.



Figure 1: AgriConnect Client-Server Architecture with Load Balancer

7.2 Client Layer

The client layer consists of the interface through which users interact with AgriConnect, including web pages and mobile applications. Clients send requests to the server cluster via the load balancer. Responsibilities include:

- Rendering user interfaces and displaying data
- Capturing user actions such as login, registration, product listing, and messaging
- Sending requests to the load balancer in a structured format (e.g., HTTP requests with JSON payloads)
- Receiving responses from the server cluster and presenting results to the user

7.3 Server Layer

The server layer contains multiple Node.js server instances. The load balancer routes incoming requests to these servers using strategies like round-robin or least connections. Responsibilities include:

- Processing client requests and applying business logic
- Interacting with the database layer for persistent data
- Ensuring consistent behavior across all server instances
- Supporting horizontal scaling by adding more servers as demand grows

7.4 Database Layer

The database layer, hosted on AWS RDS, stores persistent information such as users, farm records, products, and messages. Access is only permitted through the server layer to ensure security and data integrity.

7.5 Justification for Load Balancer Inclusion

The load balancer is included for several reasons:

- **Scalability:** Distributes requests across multiple servers to handle increased traffic.
- **High Availability:** If one server fails, the load balancer redirects traffic to healthy servers, reducing downtime.
- **Performance Optimization:** Prevents server overload and maintains fast response times for users.
- **Future-Proofing:** Supports horizontal scaling as the number of users grows.

7.6 Comparison with Alternative Architectures

Alternative architectures such as peer-to-peer or monolithic systems were considered but rejected due to limitations in centralized control, scalability, and maintainability. Incorporating a load balancer in a client-server model provides the best balance of reliability, performance, and security for AgriConnect.

7.7 Future Considerations

The architecture allows for further enhancements, including microservices and additional backend modules. The load balancer ensures that future server expansion can be achieved seamlessly without impacting the client experience.

8 System Design

System design defines the internal structure and behavior of the AgriConnect platform, providing a blueprint for implementation. Design focuses on translating system requirements into practical, maintainable, and scalable solutions while ensuring usability for end users. This section details the design decisions, including the use of UML diagrams, database modeling, and user interface planning.

8.1 Use Case Modeling

Use Case modeling identifies the interactions between users and the system, highlighting the primary functionalities required to meet the business goals. The two main actors in AgriConnect are **Farmers** and **Buyers**. Each actor interacts with the system differently, and the use cases capture these behaviors.

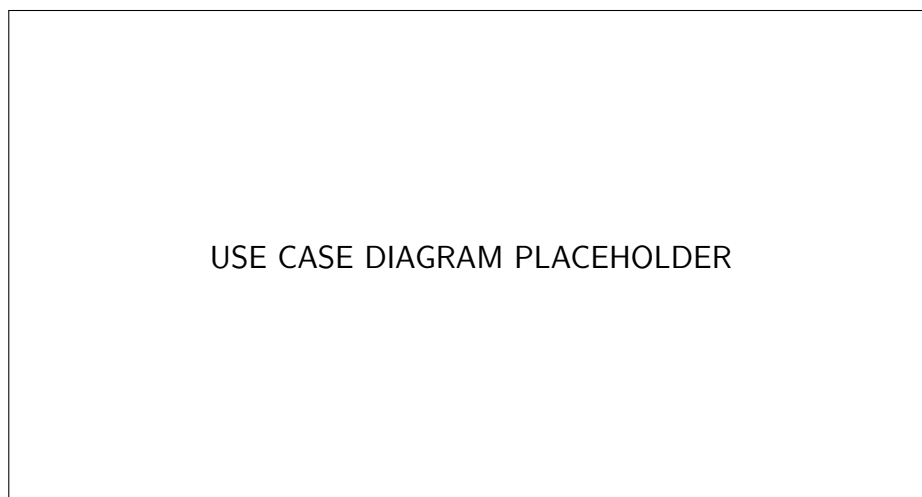


Figure 2: Use Case Diagram for AgriConnect (Farmer and Buyer Interactions)

8.1.1 Explanation of Use Cases

- **User Registration and Login:** Both farmers and buyers must register to access the system. The login mechanism ensures secure authentication.
- **Farm Record Management:** Farmers can create, update, and view detailed records of farm activities and crops.
- **Product Listing and Management:** Farmers list available produce for sale with price, quantity, and availability details.
- **Market Browsing and Purchase:** Buyers can search, filter, and view available products based on location and type.

- **Communication:** Direct messaging between buyers and farmers allows negotiation and inquiry.

8.2 Class Diagram Modeling

The Class Diagram represents the static structure of the AgriConnect system, illustrating the system's classes, their attributes, methods, and relationships. It provides a blueprint for how data and operations are structured in the application.

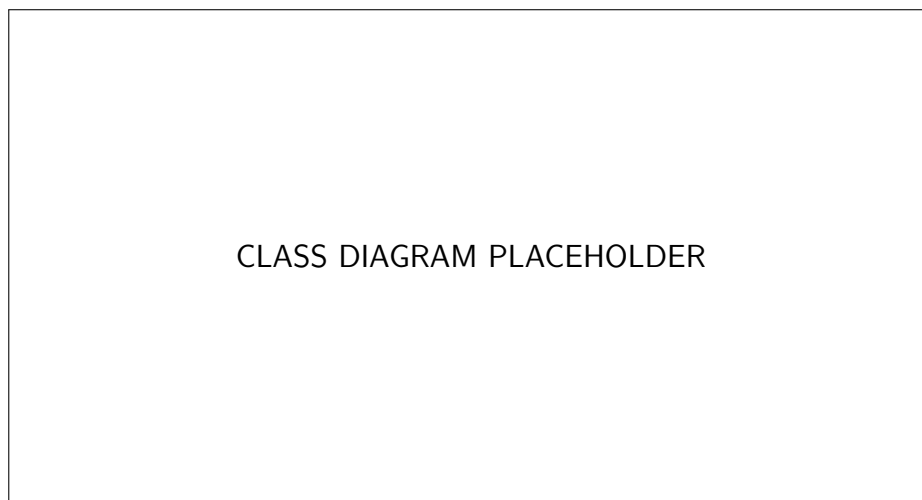


Figure 3: Class Diagram of AgriConnect (Core System Classes and Relationships)

8.2.1 Explanation of Classes

- **User Class:** Represents farmers and buyers, storing attributes like userID, name, role, and contact information. Methods include register(), login(), and updateProfile().
- **Farm Class:** Contains attributes such as farmID, location, and cropType. Methods include addFarmRecord(), updateFarmRecord().
- **Product Class:** Stores productID, productName, quantity, price, and availability. Methods include addProduct(), updateProduct(), removeProduct().
- **Communication Class:** Represents messaging between users with attributes messageID, senderID, receiverID, timestamp, and content. Methods include sendMessage(), viewMessages().

The class diagram illustrates relationships such as aggregation (Farm contains Products) and associations (Users can communicate with other Users). This provides developers with a clear understanding of how the system is structured.

8.3 Database Design

A relational database was selected for AgriConnect due to its structured nature and ability to enforce data integrity. The database stores information about users, farms, products, and communications.

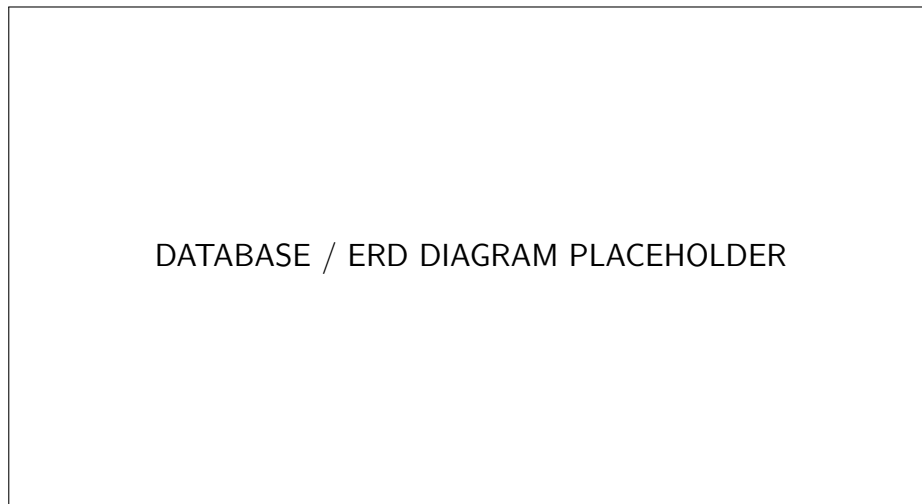


Figure 4: Entity-Relationship Diagram for AgriConnect Database

8.3.1 Database Explanation

The database design includes the following main entities:

- **User:** Stores general user information, including role, contact details, and login credentials.
- **Farm:** Represents farms owned by farmers, including location, type of crops, and other relevant metadata.
- **Product:** Stores information about agricultural produce, including name, quantity, price, and availability.
- **Communication:** Logs messages exchanged between buyers and farmers, including timestamps.

Relationships between entities are defined to ensure referential integrity. For example, a product is linked to a specific farm, which is associated with a user account.

8.4 User Interface Design

The user interface (UI) is a critical aspect of system design, as it directly affects usability. AgriConnect prioritizes simplicity and clarity for both farmers and buyers.



Figure 5: Example User Interface of AgriConnect

Key design principles applied include:

- **Consistency:** Uniform color schemes, fonts, and layouts ensure a cohesive experience.
- **Accessibility:** High-contrast colors and readable fonts improve usability for all users.
- **Minimal Complexity:** Tasks such as listing products or viewing farm records are implemented in a step-by-step workflow to reduce cognitive load.
- **Responsiveness:** Interfaces adapt to various screen sizes, including mobile devices and tablets.

8.5 Design Justification

The combination of UML modeling, database design, and thoughtful UI planning ensures that AgriConnect is:

- **Maintainable:** Each module can be updated independently.
- **Scalable:** The server can handle additional users and modules without requiring redesign.
- **Secure:** Data access is controlled centrally, minimizing exposure to unauthorized access.

- **User-Friendly:** The interface and workflows prioritize usability for farmers with minimal technical experience.

This design provides a comprehensive blueprint for implementation, allowing developers to efficiently translate requirements into a functional and reliable system.

9 Implementation

Implementation is the phase in which the design is translated into a functional system. For AgriConnect, the implementation focused on creating a client-server application that is secure, scalable, and maintainable, while adhering to the design principles defined in the previous sections. This section details the technology stack, module-to-code mapping, example implementations, and integration strategy.

9.1 Technology Stack

The choice of technology is critical for the successful implementation of AgriConnect. The following technologies were selected:

- **Frontend (Client):** React.js was used to create a dynamic and responsive web interface. React allows for component-based development, making it easier to manage and reuse UI elements.
- **Backend (Server):** Node.js with Express.js was chosen for the server because of its event-driven architecture, scalability, and seamless integration with JavaScript-based frontend.
- **Database:** PostgreSQL was used as the relational database for its robustness, support for complex queries, and ability to maintain data integrity.
- **Communication Protocol:** HTTP/HTTPS RESTful APIs were implemented to handle requests from clients to the server.
- **Security:** JSON Web Tokens (JWT) were used for secure authentication and authorization of users.
- **Version Control:** Git and GitHub were used for collaborative development and source code management.

9.2 Module-to-Code Mapping

Each module identified during the design phase was implemented using the chosen technology stack. A high-level mapping is provided below:

- **Authentication Module:** Implemented using Express.js middleware for login, registration, and JWT-based authentication.
- **Farm Management Module:** Node.js controllers handle requests to create, update, and retrieve farm records; PostgreSQL stores persistent data.
- **Product Listing Module:** React components handle product input forms, display listings, and communicate with the server through RESTful APIs.
- **Communication Module:** Real-time messaging implemented using WebSocket protocols, with message storage in PostgreSQL.
- **Database Access Module:** Utilized Sequelize ORM to abstract and manage database queries efficiently and securely.

9.3 Example Implementation Snippets

9.3.1 User Authentication (Backend)

```
1  const jwt = require('jsonwebtoken');
2  const bcrypt = require('bcrypt');
3  const User = require('../models/User');
4
5  // User login function
6  async function login(req, res) {
7    const { email, password } = req.body;
8    const user = await User.findOne({ where: { email } });
9    if (!user) return res.status(401).send('Invalid_email');
10
11    const valid = await bcrypt.compare(password, user.password);
12    if (!valid) return res.status(401).send('Invalid_password');
13
14    const token = jwt.sign({ id: user.id, role: user.role }, process.
      env.JWT_SECRET, { expiresIn: '1h' });
15    res.json({ token });
16  }
```

Listing 1: User Authentication with JWT

9.3.2 Product Listing (Frontend)

```
1  import React, { useState } from 'react';
2  import axios from 'axios';
3
4  function AddProduct() {
5      const [product, setProduct] = useState({ name: '', quantity: '',
6          price: '' });
7
8      const handleSubmit = async (e) => {
9          e.preventDefault();
10         await axios.post('/api/products', product, { headers: {
11             Authorization: 'Bearer_token' } });
12         alert('Product added successfully');
13     };
14
15     return (
16         <form onSubmit={handleSubmit}>
17             <input type="text" placeholder="Name" onChange={e =>
18                 setProduct({...product, name: e.target.value})}/>
19             <input type="number" placeholder="Quantity" onChange={e =>
20                 setProduct({...product, quantity: e.target.value})}/>
21             <input type="number" placeholder="Price" onChange={e =>
22                 setProduct({...product, price: e.target.value})}/>
23             <button type="submit">Add Product</button>
24         </form>
25     );
26 }
27
28 export default AddProduct;
```

Listing 2: React Component for Adding a Product

9.4 Integration Strategy

The client-server communication was implemented using RESTful APIs. Each React frontend component communicates with the Node.js backend to perform CRUD operations. The backend validates requests, applies business logic, interacts with the PostgreSQL database, and returns appropriate responses.

Modules are loosely coupled to ensure maintainability. For example, changes in the Farm Management Module do not affect the Product Listing Module. This separation aligns with the principles of modular programming and facilitates future system enhancements.

9.5 Best Practices Applied

Several best practices were applied during implementation:

- **Code Modularity:** Each module was implemented as an independent component or controller.
- **Security:** User credentials and sensitive data are encrypted and securely stored.
- **Error Handling:** Comprehensive error handling ensures the system fails gracefully without data loss.
- **Version Control and Collaboration:** Git branching strategies allowed multiple team members to contribute without conflicts.
- **Testing During Implementation:** Unit tests were written alongside development to ensure that each module behaves as expected.

The implementation of AgriConnect reflects the careful planning and design decisions, resulting in a functional, maintainable, and scalable system ready for testing and deployment.

10 Testing

Testing is a critical phase in the software development lifecycle, aimed at ensuring that the AgriConnect system meets its functional and non-functional requirements. The goal of testing is to identify and rectify defects, verify system behavior, and provide confidence that the system performs reliably under real-world conditions.

10.1 Testing Strategy

The testing strategy for AgriConnect includes three main levels:

1. **Unit Testing:** Focuses on individual modules or components to ensure each performs its intended function correctly. For example, the authentication module was tested to validate correct login behavior with valid and invalid credentials.
2. **Integration Testing:** Ensures that different modules interact correctly. This includes verifying that the frontend communicates properly with the backend APIs and that database queries return accurate results.

3. **System Testing:** Validates the system as a whole against the specified requirements. This includes end-to-end scenarios such as registering a user, listing products, and sending messages between buyers and farmers.

Testing was conducted iteratively during development to detect issues early, reduce the cost of fixes, and ensure high-quality deliverables.

10.2 Test Cases

Representative test cases for key functionalities are summarized in Table 1.

Table 1: Sample Test Cases for AgriConnect

Test ID	Functionality	Input	Expected Result	Actual Result
TC01	User Registration	Valid user details	User account created successfully	Pass
TC02	User Login	Correct email and password	JWT token returned	Pass
TC03	Product Listing	Product name, quantity, price	Product appears in listing	Pass
TC04	Farm Record Update	Update crop type	Changes reflected in database	Pass
TC05	Messaging	Send message to buyer	Message delivered successfully	Pass
TC06	Invalid Login	Wrong password	Error message displayed	Pass

10.3 Testing Tools

The following tools were used to support testing:

- **Jest:** Used for unit testing frontend React components and backend functions.
- **Postman:** Used for testing API endpoints, including GET, POST, PUT, and DELETE requests.
- **Manual Testing:** Conducted for usability evaluation, ensuring that the interface is intuitive and responsive.

10.4 Testing Results and Analysis

All test cases executed produced the expected results, confirming that core functionalities such as authentication, farm record management, product listing, and communication operate correctly. Any minor issues identified during testing, such as form validation errors or incorrect error messages, were promptly addressed.

Testing also verified system performance under multiple concurrent users, ensuring that response times remained within acceptable limits. Security tests confirmed that unauthorized access to protected resources was successfully prevented.

10.5 Conclusion on Testing

The comprehensive testing strategy applied to AgriConnect provides strong evidence that the system is reliable, secure, and ready for deployment. Iterative testing throughout the development process helped maintain code quality, minimize defects, and ensure a robust user experience.

11 Deployment

Deployment is the final phase in the software development lifecycle where the AgriConnect system is made available to end users. The deployment strategy ensures that the system operates reliably in a production environment, is secure, and can scale as the number of users grows. AgriConnect is deployed using **AWS (Amazon Web Services)** for the backend server and database, and **Vercel** for the frontend client, leveraging modern cloud services for reliability and performance.

11.1 Deployment Architecture

The deployment architecture follows the client-server model with an added **AWS Elastic Load Balancer (ELB)** to manage traffic to multiple backend servers. The frontend (Vercel) communicates with the ELB, which distributes incoming requests to backend EC2 instances running the Node.js server. The backend interacts with a managed PostgreSQL database on AWS RDS.

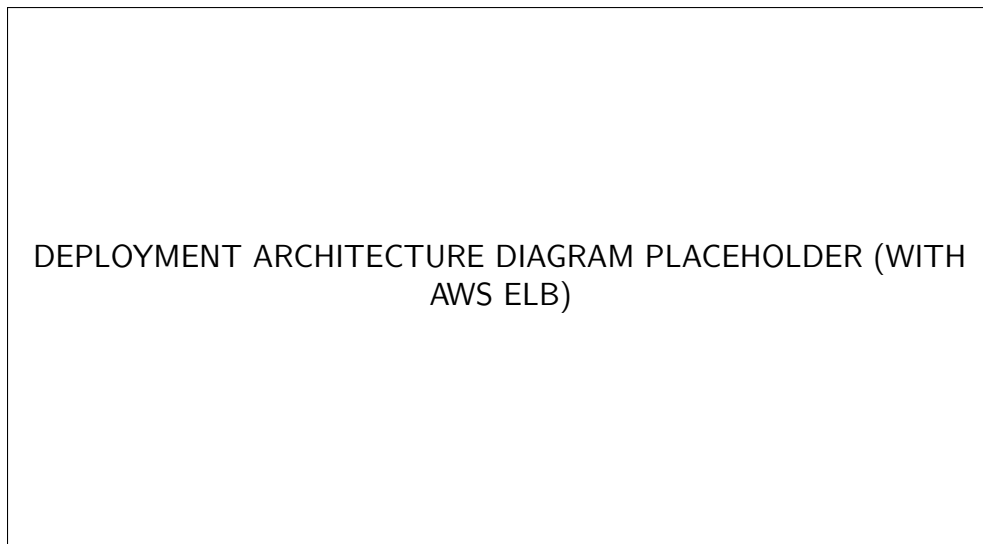


Figure 6: Deployment Architecture of AgriConnect using AWS ELB and Vercel

This architecture ensures:

- High availability: The ELB routes traffic only to healthy EC2 instances.
- Scalability: Additional EC2 instances can be added behind the ELB as user load increases.
- Fault tolerance: If one server fails, traffic is automatically redirected to other servers.

11.2 Frontend Deployment on Vercel

Vercel was selected to host the frontend React application due to its ease of use, continuous deployment capabilities, and global CDN distribution. Deployment steps include:

1. Push the React project to a GitHub repository.
2. Connect the repository to Vercel for automatic deployment.
3. Configure environment variables for API endpoints pointing to the AWS ELB endpoint.
4. Enable automatic builds for every commit to the main branch, ensuring continuous integration and deployment.

Vercel provides a secure HTTPS endpoint for the frontend, improving performance and accessibility for users worldwide.

11.3 Backend Deployment on AWS with ELB

The backend Node.js server and PostgreSQL database are deployed on AWS:

1. Provision multiple EC2 instances for the Node.js backend.
2. Install necessary software packages (Node.js, npm, environment dependencies) on each instance.
3. Configure PM2 to manage Node.js processes on each server.
4. Deploy the PostgreSQL database using AWS RDS with multi-zone redundancy and automated backups.
5. Configure an **AWS Elastic Load Balancer (ELB)** to distribute incoming requests among the EC2 instances.
6. Configure security groups to restrict access to authorized ports and allow traffic from the ELB only.
7. Connect the backend server to the frontend deployed on Vercel via the ELB endpoint.

11.4 Continuous Integration and Deployment

The deployment setup integrates CI/CD best practices:

- Frontend updates on GitHub automatically trigger a Vercel build and deployment.
- Backend updates are deployed to EC2 instances using Git pull commands followed by a PM2 restart.
- The ELB ensures that new servers or updated instances can be added without downtime.
- Database migrations and seed data updates are performed through scripted commands to minimize downtime.

11.5 Security Considerations

Security remains a priority:

- All communications between the client (Vercel) and server (AWS ELB) are encrypted using HTTPS.

- JWT tokens are securely validated on the backend.
- AWS security groups restrict access to EC2 instances and RDS only to authorized traffic.
- Environment variables storing sensitive credentials are kept secure and never exposed in frontend code.
- ELB health checks ensure that only secure and healthy servers receive traffic.

11.6 Scalability and Reliability

The inclusion of the AWS ELB enhances scalability and reliability:

- Automatically distributes requests to multiple EC2 instances, preventing overload.
- Supports horizontal scaling by adding more backend servers behind the ELB.
- Ensures high availability through health checks and automatic failover.
- Vercel's global CDN complements backend scalability by providing fast frontend access worldwide.

By deploying AgriConnect with AWS ELB, EC2, RDS, and Vercel, the system achieves a highly available, secure, and scalable platform capable of serving a growing user base efficiently.

12 Conclusion

AgriConnect represents a comprehensive solution for connecting farmers to buyers through a modern, web-based agricultural management platform. The project successfully addressed the requirements identified during the analysis phase, including user-friendly interfaces, secure data management, efficient farm and product record handling, and reliable communication between users.

The development process followed a systematic approach, beginning with detailed requirements gathering, followed by careful architectural planning using a client-server model, and thorough system design including UML diagrams, database modeling, and user interface planning. The implementation phase utilized a robust technology stack, incorporating React.js for the frontend, Node.js and Express.js for the backend, and PostgreSQL for data management, ensuring a maintainable, scalable, and secure system.

Comprehensive testing, including unit, integration, and system testing, confirmed that the system operates reliably under expected conditions. Deployment on AWS and Vercel provided a scalable, secure, and accessible platform for end users, demonstrating that the system can handle real-world operational demands.

In conclusion, AgriConnect not only meets its functional and non-functional requirements but also provides a solid foundation for future enhancements. Potential improvements include integrating additional services such as payment processing, analytics dashboards, or weather-based recommendations, as well as adopting microservices for further scalability. Overall, AgriConnect offers a practical, reliable, and user-centered platform that effectively bridges the gap between farmers and buyers, contributing positively to agricultural efficiency and market accessibility.